1.1.10. Depth-Fir… 13:06

Write a C program to implement and perform a Depth-First Search (DFS) on a directed graph represented using an adjacency list. The program should print all vertices reachable from a given starting vertex in the order they are visited by DFS.



Input Format:

- The first line contains two integers: V (number of vertices) and E (number of edges).
- The next E lines each contain two integers: u and v, where there is a directed edge from node u to node v.
- The last line contains a single integer, which is the starting node for the DFS traversal.

Output Format:

- Print the vertices reachable from the starting vertex in the exact order they are visited by DFS.
- Each vertex should be printed, followed by a single space.
- The output should be printed on one line only as:

```
v1 v2 v3 ... vn
```

Constraints:

- $1 \leq V \leq 100$
- $0 \leq E \leq V \times (V-1)$
- Vertices are numbered from 0 to $V-1$

Note:

- In DFS, the smaller numbered node is visited first when there are

Explorer

dfs_direc…

Submit

Debugger

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 100

// Graph represented using adjacency list
typedef struct Node {
    int vertex;
    struct Node* next;
} Node;

Node* adjList[MAX_VERTICES];
int visited[MAX_VERTICES];

// Function to create a new adjacency list node
Node* createNode(int vertex) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = vertex;
    newNode->next = NULL;
    return newNode;



}

// Function to add an edge to the graph
```

‹ Prev Reset Submit Next ›

multiple adjacent nodes, ensuring a consistent traversal order.

- Refer to the visible test cases for a better understanding.

Sample Test Cases +

```
31  void addEdge(int u, int v) {
32      Node* newNode = createNode(v);
33      newNode->next = adjList[u];
34      adjList[u] = newNode;
35
36
37
38
39
40
41  }
42
43  // Function to sort the adjacency list for each vertex
44  void sortAdjList(int V) {
45      for (int i = 0; i < V; i++) {
46          Node* curr = adjList[i];
47          if (!curr) continue;
48          int swapped;
49          do {
50              swapped = 0;
51              Node* ptr = adjList[i];
52              while (ptr->next != NULL) {
53                  if (ptr->vertex > ptr->next->vertex) {
54                      int temp = ptr->vertex;
55                      ptr->vertex = ptr->next->vertex;
```

```
56      ptr->next->vertex =
        temp;
57
58      swapped = 1;
                    }
59                  ptr =
60      ptr->next;
                }
61          } while
62      (swapped);
63      }
64
65
66
67
68
69
70      }

71  v   // Depth-First Search
72      (DFS) function
73      void DFS(int start) {
            visited[start] = 1;
74          printf("%d ",
        start);
75  v       Node* temp =
        adjList[start];
76          while (temp !=
        NULL) {
                int
77  v       connectedVertex = temp-
        >vertex;
                if
78      (!visited[connectedVert
        ex]) {
79
80      DFS(connectedVertex);
                }
81              temp = temp-
82      >next;
83          }
84
```

```
86
87
88
89
90  }
91
92  int main() {
93      int V, E;
94      int u, v, start;
95      // Read number of
    vertices and edges
96      scanf("%d %d", &V,
97  &E);
98      // Initialize
    adjacency list
99      for (int i = 0; i
    < V; i++) {
100         adjList[i] =
101 NULL;
102         visited[i] = 0;
103     }
104
    // Read the edges
105     for (int i = 0; i
    < E; i++) {
106         scanf("%d %d",
107 &u, &v);
108         addEdge(u, v);
109     }
    // Sort the
110 adjacency list for
111 each vertex
112     sortAdjList(V);
113     // Read the
    starting node
114     scanf("%d",
115 &start);
    // Perform DFS
```